CS2201 Computer Organization & Architecture

L20: Block Replacement Techniques



John Jose

Associate Professor

Department of Computer Science & Engineering

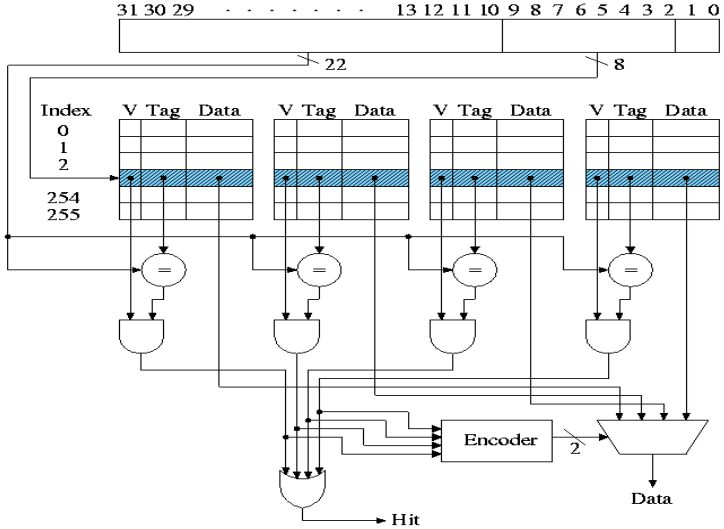
Indian Institute of Technology Guwahati

Four cache memory design choices

- Where can a block be placed in the cache?
  - Block Placement
- How is a block found if it is in cache memory?
  - Block Identification
- Which block should be replaced on a miss?
  - Block Replacement
- What happens on a write?
  - Write Strategy

Block Replacement

- Cache has finite size. What do we do when it is full?
- Direct Mapped is Easy
- Which block to be replaced for a set associative cache?



Block Replacement Algorithms

- Random
- First In First Out (FIFO)
- Last In First Out (LIFO)
- Least Recently Used (LRU)
- Pseudo-LRU (PLRU)
- Not Recently Used (NRU)
- Least Frequently Used (LFU)
- Re-Reference Interval Prediction (RRIP)
- Optimal

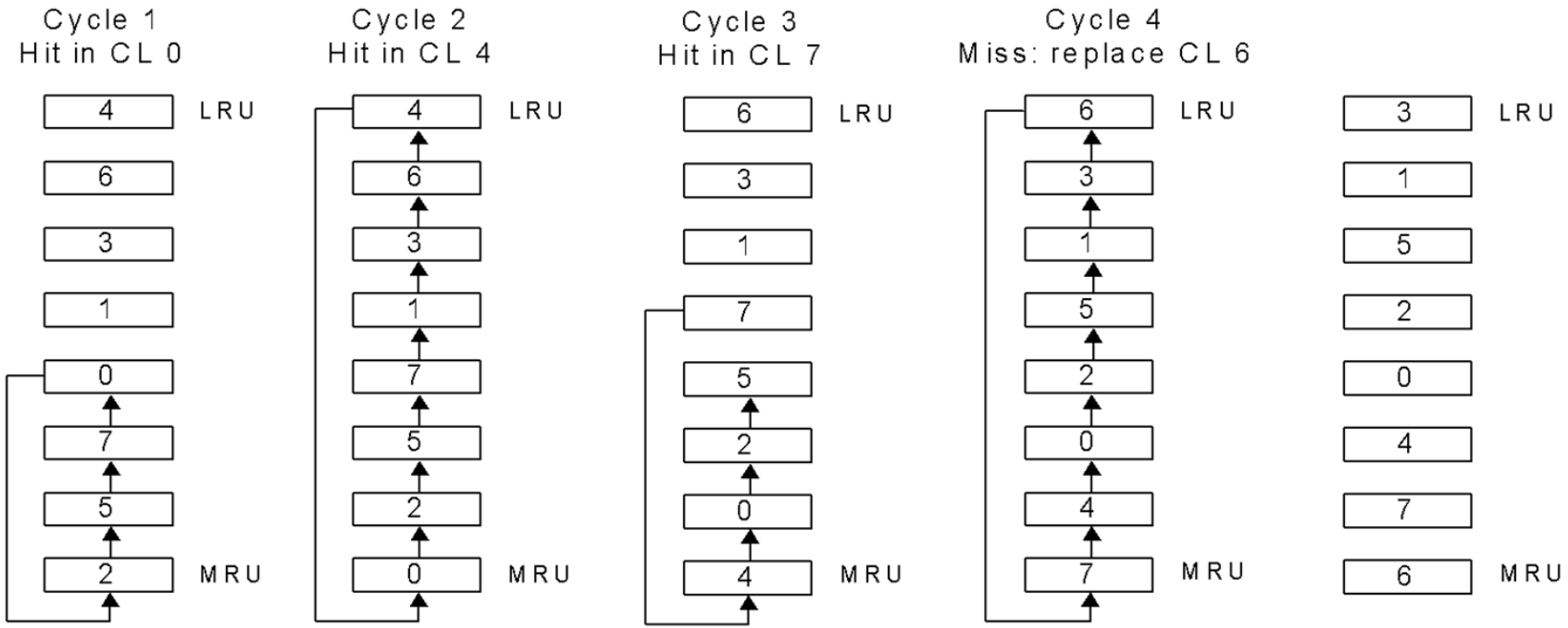
Random & FIFO Replacement Policy

- **Random policy** needs a pseudo-random number generator
- Makes no attempt to take advantage of any temporal or spatial localities
- **First-in, First-out(FIFO) policy** evict the block that has been in the cache the longest
- It requires a queue Q to store references
- Blocks are enqueued in Q, dequeue operation on Q to determine which block to evict.

Least-Recently Used

- For associativity =2, LRU is equivalent to NMRU
  - Single bit per line indicates LRU/MRU
  - Set/clear on each access
- For a>2, LRU is difficult/expensive
  - Record Timestamps? How many bits?
  - Must find min timestamp on each eviction
  - Sorted list? Re-sort on every access?

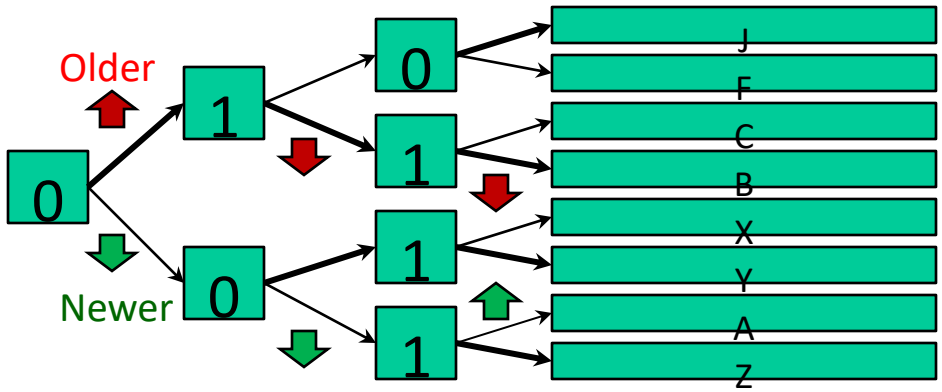
LRU Implementation



Optimal Replacement Policy

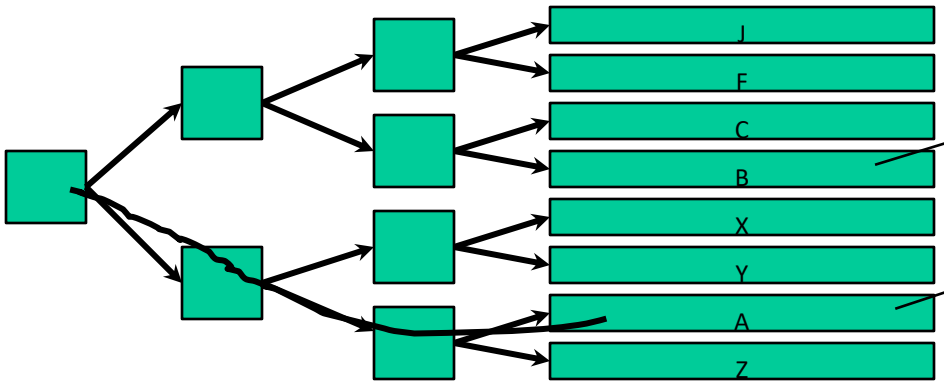
- Evict block with longest reuse distance
  - i.e. next reference to block is farthest in future
  - Requires knowledge of the future!
- Can't build it, but can model it with trace
- Useful, since it reveals opportunity
- Optimal better than LRU
  - (X,A,B,C,D,X): LRU 4-way SA cache, 2nd X will miss

Practical Pseudo-LRU



- Rather than true LRU, use binary tree
- Each node records which half is older/newer
- Update nodes on each reference
- Follow older pointers to find LRU victim

Practical Pseudo-LRU



J Y X Z B C F A

011: PLRU Block B is here

110: MRU block is here

Partial Order Encoded in Tree:

Z < A

Y < X

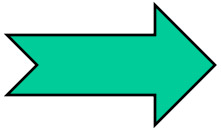
B < C

J < F

A > X

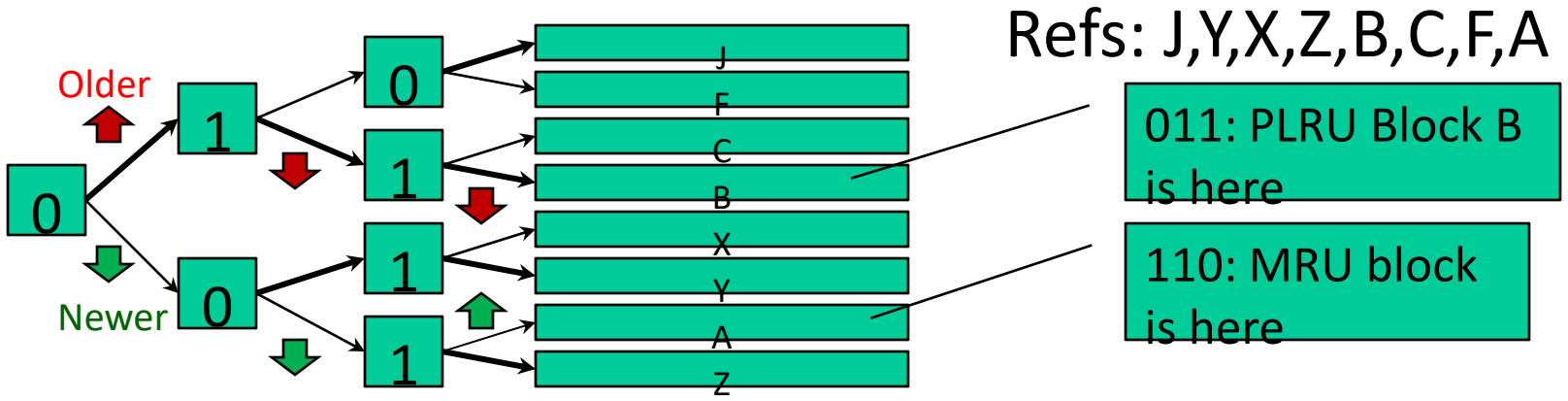
C < F

A > F



B	C	F	A
	J		
	Y	X	
		Z	

Practical Pseudo-LRU



- Binary tree encodes PLRU partial order
- At each level point to LRU half of subtree
- Each access: flip nodes along path to block
- Eviction: follow LRU path
- Overhead: *(a-1)/a* bits per block

Not Recently Used (NRU)

- Keep NRU state in 1 bit/block
  - Bit is reset to 0 when installed / re referenced
  - Bit is set to 1 when it is not referenced and other block in the same set is referenced
  - Evictions favor NRU=1 blocks
  - If all blocks are NRU=0 / 1 then pick by random

Re-reference Interval Prediction

- RRIP
- Extends NRU to multiple bits
  - Start in the middle
  - promote on hit
  - demote over time
- Can predict near-immediate, intermediate, and distant re-reference

Least Frequently Used

- Counter per block, incremented on reference
- Evictions choose lowest count
- Storage overhead and sorting overhead



johnjose@iitg.ac.in

http://www.iitg.ac.in/johnjose/

https://www.linkedin.com/in/john-jose-bb8431106/